

Diffusion Lecture 2

Motivation

• images come from a complex distribution.

→ we sample from Gaussian noise and go towards images distribution.

• Gradient without the log is intractable because

1. PDFs sum to 1 and the normalizing constant is intractable

2. Some places have very low values.
Numerically unstable

• Gradient of log probability

$$1. \quad \nabla_x \log(p_{\text{data}}(x)) = \nabla_x \log(f(x)) - \nabla_x \log(Z) = 1 \\ = \nabla_x \log(f(x))$$

$$2. \quad \nabla_x \log(p) = \frac{\nabla p}{p} \rightarrow \text{points in the same direction as } p$$

3. more numerically stable.

$$\text{Score function: } s(x) = \nabla_x \log p(x).$$

• when further away, score magnitude is bigger.

When we are going to the higher density regions,
we want to inject noise.

$$\text{Langevin Sampling: } x_t = x_{t-1} + \frac{\alpha}{2} \nabla_x \log p_{\text{data}}(x_{t-1}) + \sqrt{\alpha} \epsilon_t.$$

↳ has a stochastic term for exploration (MCMC).

Score Estimation

Score matching

$$\mathcal{L}_{SM} = \mathbb{E}_x \left[\| s_\theta(x) - \nabla_x \log p_{data}(x) \|^2 \right]$$

↑
we do not have access here.

- implicit score matching (ISM): integrate by parts

$$\mathcal{L}_{ISM} = \mathbb{E}_x \left[\frac{1}{2} \| s_\theta(x) \|^2 + \nabla_x \cdot s_\theta(x) \right]$$

- sliced score matching (SSM): project on vectors

$$\mathcal{L}_{SSM} = \mathbb{E}_{x_0, \epsilon, \epsilon', v} \left[2v^T \nabla_x s_\theta(x) v + |v^T s_\theta(x)|^2 \right]$$

- Denoising score matching

for $x \sim N(\mu, \sigma^2)$

pdf:
$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

score is:
$$s(x) = -\frac{x-\mu}{\sigma^2} \quad \text{score of a gaussian}$$

Idea: add noise to data and use analytical expression of score

$$\tilde{x} = x + \epsilon \quad \text{with} \quad \epsilon \sim N(0, 1)$$

therefore
$$q_\epsilon(\tilde{x} | x) = N(x, \epsilon^2 I)$$

$$\rightarrow \nabla_{\tilde{x}} \log q_\epsilon(\tilde{x} | x) = -\frac{\tilde{x} - x}{\epsilon^2}$$

derivation: leverage expression of score for a gaussian distribution.

- add noise to the distribution.

$$q_{\theta}(\tilde{x}) = \int q_{\theta}(\tilde{x}|x) p_{\text{data}}(x) dx \quad \text{noise distribution}$$

$$q_{\theta}(\tilde{x}, x)$$

similar to a mixture of Gaussians.

- estimating the score matching objective on the noised distribution

Definition of score matching of q_{θ}

$$\mathcal{L}_{\text{SM}}(q_{\theta}) = \mathbb{E}_{\tilde{x}} [\|s_{\theta}(\tilde{x}) - \nabla_{\tilde{x}} \log q_{\theta}(\tilde{x})\|^2]$$

we can show this is equal to

$$\mathcal{L}_{\text{SM}}(q_{\theta}) = \mathbb{E}_{\tilde{x}, x} [\|s_{\theta}(\tilde{x}) - \underbrace{\nabla_{\tilde{x}} \log q_{\theta}(\tilde{x}|x)}_{-\frac{\tilde{x}-x}{\sigma^2} \text{ tractable}}\|^2]$$

derivation: (original denoising diffusion paper).

$$(1) \quad \mathbb{E}_{\tilde{x}} [\|s_{\theta}(\tilde{x}) - \nabla_{\tilde{x}} \log q_{\theta}(\tilde{x})\|^2]$$

$$(2) \quad \mathbb{E}_{\tilde{x}, x} [\|s_{\theta}(\tilde{x}) - \nabla_{\tilde{x}} \log q_{\theta}(\tilde{x}|x)\|^2]$$

$$\|a-b\|^2 = \|a\|^2 - 2\langle a, b \rangle + \|b\|^2$$

we want to optimize w.r.t θ .

b term is not a function of $\theta \rightarrow$ we don't care.

a terms are the same in (1) and (2)

we only show $-2\langle a, b \rangle$ terms are equal.

Denoising Score matching: tractable loss.

Limitations of vanilla DSM.

q_{θ} close to P_{data} but poor estimation in low density region.

$$\mathcal{L} = \int \|\mathbf{s}_{\theta}(\tilde{\mathbf{x}}) - \nabla \log q_{\theta}(\tilde{\mathbf{x}})\|^2 q_{\theta}(\tilde{\mathbf{x}}) d\tilde{\mathbf{x}}$$

- low density regions' score estimations are bad.

bias-variance tradeoff (simple)

- we can add more noise to increase density, but this is not desirable because now the distribution is too different from data distribution and is difficult to learn.

Varying Noise Level.

idea: estimate score at different noise levels $\sigma_1 < \dots < \sigma_T$.

$$\mathbf{s}_{\theta}(\mathbf{x}) \longrightarrow \mathbf{s}_{\theta}(\mathbf{x}, \sigma_i)$$

Loss: Noise Conditional Score Network.

$$\mathcal{L}_{NCSN} = \sum_{i=1}^T \lambda(i) E_{\mathbf{x}} [\|\mathbf{s}_{\theta}(\mathbf{x}, \sigma_i) - \nabla_{\mathbf{x}} \log q_{\sigma_i}(\mathbf{x})\|_2^2]$$

ALD: Annealed Langevin Dynamics (reducing noise).

Sampling with ALD:

1 Sample Noise

$$2 \quad \mathbf{x} \longleftarrow \mathbf{x} + \frac{\alpha_i}{2} \mathbf{s}_{\theta}(\mathbf{x}, \sigma_i) + \sqrt{\alpha_i} \epsilon$$

3 final image $\mathbf{x}_0$.

Parallel between DDPM and NCSN.

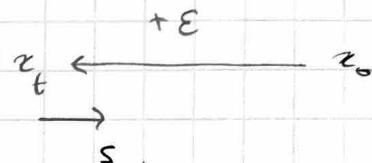
DDPM:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

$$q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I)$$

$$\nabla_{x_t} \log(q(x_t | x_0)) = - \frac{\epsilon}{\sqrt{1 - \bar{\alpha}_t}}$$

• See ϵ equal to - noise added.



$$\nabla \log q = - \frac{(x - \mu)}{\sigma^2}$$

$$= \frac{\sqrt{1 - \bar{\alpha}_t} \epsilon}{(1 - \bar{\alpha}_t)} = - \frac{\epsilon}{\sqrt{1 - \bar{\alpha}_t}}$$

DDPM: variance preserving

NCSN: variance exploding.

Differential Formulation

idea - continuous equivalent of increments & add gaussian noise

Wiener Process

- stochastic process. Family of random variables indexed by t (time)
- Wiener process follows
 - $W_0 = 0$
 - $W_t - W_s \sim N(0, (t-s)I)$
 - independent increments (not the values).

DDPM

$$x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \epsilon.$$

$$x_t - x_{t-1} = \left[\sqrt{1 - \beta_t} - 1 \right] x_{t-1} + \sqrt{\beta_t} \epsilon.$$

define $\beta_t = \underbrace{\beta(t) dt}_{\text{rate of noise at time } t}$

$$x_t - x_{t-1} = \left[\sqrt{1 - \beta(t) dt} - 1 \right] x_{t-1} + \sqrt{\beta(t) dt} \epsilon.$$

as $t \rightarrow 0$, what happens: $x_t - x_{t-1} = dx$

as $dt \rightarrow 0$, Taylor exp $\Rightarrow$

$$dx \sim \left[1 - \frac{1}{2} \beta(t) dt - 1 \right] x + \sqrt{\beta(t)} dW.$$

$$dW = \sqrt{dt} \epsilon.$$

$$\hookrightarrow dx = -\frac{1}{2} \beta x dt + \sqrt{\beta} dW.$$

$$dx = \underbrace{f(x,t) dt}_{\text{drift}} + \underbrace{g(t) dW}_{\text{diffusion}}$$

SDE Variants.

① Variance Preserving

DDPM

$$f(x,t) = -\frac{1}{2} \beta(t) x$$

$$g(t) = \sqrt{\beta(t)}$$

② Variance Exploding

NCSN

$$f(x,t) = 0$$

$$g(t) = \sqrt{\frac{d[\sigma^2(t)]}{dt}}$$

Training denoising into clean images.

$$\mathcal{L}_{\text{psm}} = \mathbb{E}_{t, x_0, x_t} \left[\lambda_t \left\| s_\theta(x_t, t) - \underbrace{\nabla_{x_t} \log p(x_t | x_0)}_{-\frac{\epsilon}{\sigma_t}} \right\|^2 \right]$$

$$x_t = \alpha_t x_0 + \sigma_t \epsilon$$

DDPM Variance preserving: $x_t | x_0 \sim N(\sqrt{\alpha_t} x_0, (1 - \alpha_t) I)$

NCSN Variance exploding: $x_t | x_0 \sim N(x_0, \sigma_t^2 I)$

1 Sample $x \sim p_{\text{data}}, \epsilon \sim N(0, I), t \sim \mathcal{U}(0, T)$

$$x_t = \alpha_t x_0 + \sigma_t \epsilon$$

2 use x_t and t to predict $-\frac{\epsilon}{\sigma_t}$ via $s_\theta(x_t, t)$.

3 back prop $\mathcal{L} = \lambda_t \|s_\theta(x_t, t) + \frac{\epsilon}{\sigma_t}\|^2$ through s_θ .

Introducing the reverse formula.

Forward : Perturb data into noise

$$dx = \underbrace{f(x, t) dt}_{\text{drift}} + \underbrace{g(t) dW}_{\text{diffusion}}$$

Reverse : Transform data into noise

$$dx = \left[f(x, t) - \underbrace{g(t)^2 \nabla_x \log(p_t(x))}_{\text{correction to drift}} \right] dt + g(t) d\bar{W}$$

• Fokker-Planck equation.

* need to go towards regions of high density even more to compensate for diffusion

Inference recipe

1 sample $x_T \sim N(0, \sigma_T^2, I)$

2 Euler-Maruyama to go through reverse SDE

$$x_{t_{i-1}} = x_{t_i} + \left[f(x_{t_i}, t_i) - g(t_i)^2 \nabla_{\theta} \log(p_{t_i}(x_{t_i})) \right] \Delta t + g(t_i) \sqrt{\Delta t} \frac{\epsilon_i}{\sigma_{t_i}}$$

→ numerically solves the reverse process

Limitations of an SDE

stochastic term $\rightarrow$ slower solver

more sources of error $\rightarrow$ discretization error and injected noise

Forward SDE $\rightarrow$ Fokker-Planck $\rightarrow$ continuity $\rightarrow$ PDE-ODE.
open

$\rightarrow$ Probability Flows.

Complexity metric : Number of Function Evaluations.

Problem Statement

$$dx = \mu(x, t) dt \quad \text{with } x(t_0) = x_0.$$

DPM-Solver = Diffusion Probabilistic Model-Solver.

different levels of discretization.